

VERSION 1.1
18 JANUARI 2025



[PENGGALIAN DATA]

MODUL 2 - EDA (Exploratory Data Analysis) &
Data Visualization

DISUSUN OLEH:
ALVIYA LAELA LESTARI
KIARA AZZAHRA

DIAUDIT OLEH:
YUFIS AZHAR, S.KOM, M.KOM

LAB. INFORMATIKA
UNIVERSITAS MUHAMMADIYAH MALANG

[PENGALIAN DATA]

PERSIAPAN MATERI

Praktikan mempelajari konsep dasar EDA dan visualisasi data menggunakan python

TUJUAN

1. Mahasiswa mampu memahami konsep dan langkah - langkah dalam Exploratory Data Analysis (EDA)
2. Mahasiswa mampu memahami visualisasi data

PERSIAPAN SOFTWARE

1. **Hardware**
Komputer / Laptop
2. **Software**
Google Colaboratory, PyCharm, Jupyter Notebook
3. **Dataset (pada penjelasan materi)**
[Fastfood.csv](#)

Notebook [Modul 2.ipynb](#)

A. EXPLORATORY DATA ANALYSIS (EDA)

Data merupakan sekelompok informasi atau fakta mentah yang dapat berupa simbol, angka, kata-kata, atau citra. Namun, data yang mentah akan sulit dipahami tanpa proses analisis yang memadai. **Exploratory Data Analysis (EDA)** adalah metode eksplorasi yang digunakan untuk memahami isi dataset melalui ringkasan statistik dan visualisasi grafis. EDA bertujuan untuk menemukan pola, tren, dan hubungan antar variabel. Sehingga, data dapat dievaluasi secara mendalam untuk memberikan wawasan yang berguna sebelum melanjutkan ke tahap analisis lebih lanjut atau membangun model prediktif.

Visualisasi data merupakan salah satu tahap penting dalam EDA, yaitu proses menampilkan data dalam bentuk grafik atau diagram agar lebih mudah dipahami. Visualisasi data membantu untuk mengetahui pola yang tersembunyi dalam angka-angka mentah, sehingga user dapat melihat tren dan hubungan secara lebih intuitif. Beberapa jenis grafik yang sering digunakan meliputi histogram untuk distribusi data, bar chart untuk perbandingan kategori, dan pie chart untuk proporsi data. Library yang memudahkan kita pengolahan data dan visualisasi data diantaranya adalah **Matplotlib**, **Seaborn** dan **Pandas**.

1. Dataset

Dataset adalah kumpulan data yang diatur secara terstruktur dan disimpan dalam format tertentu, seperti tabel atau file. Dataset dapat berisi data numerik, teks, gambar, atau gabungan dari semuanya. Kalian bisa collecting data secara manual atau mengakses dataset dari platform online seperti [Kaggle](#), [UCI Machine Learning Repository](#), [Google Dataset Search](#) dan lainnya.

A. Inisialisasi Data Menggunakan Library Pandas

Dalam Python, inisialisasi data dapat dilakukan dengan berbagai cara tergantung pada jenis dan struktur data yang diperlukan. Salah satu caranya adalah inisialisasi dataset menggunakan library pandas pada python. Supaya lebih interaktif dan kalian bisa bereksperimen di materi modul 2, [akses dataset](#) berikut kemudian unduh file **fastfood.csv** dan upload ke colab kalian. Berikut merupakan cara membaca dataset dari csv file:

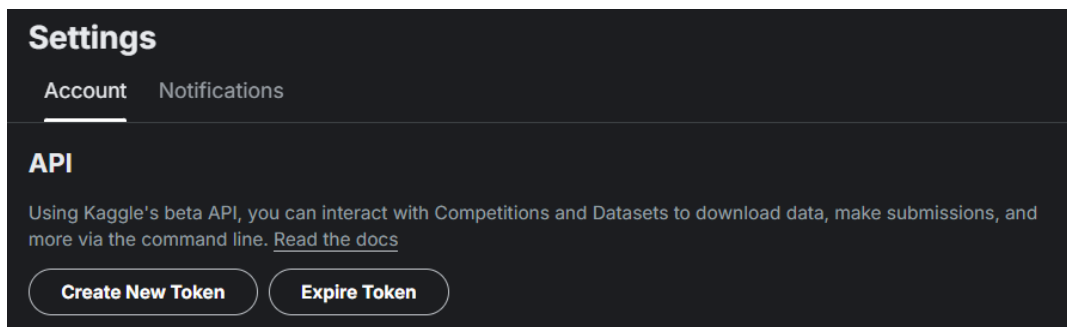
```
[ ] import pandas as pd

# Inisialisasi DataFrame dari file CSV
df = pd.read_csv('fastfood.csv') # Nama file dataset
```

B. Inisialisasi Data Dari Kaggle (Opsional)

Seperti yang sudah dijelaskan sebelumnya, dataset dapat diakses dari platform online, salah satunya adalah Kaggle. Kaggle merupakan platform online yang menyediakan sumber daya untuk ilmu data dan pembelajaran mesin (*machine learning*). Platform ini memungkinkan pengguna untuk menemukan, menerbitkan, dan menggunakan kumpulan data, serta mengikuti kompetisi sains data. Dataset dapat bervariasi jumlah dan tipe datanya. Akan sangat memakan storage dan waktu jika kita selalu mengunduh dataset dari

platform online seperti kaggle. Tapi tenang saja, kita dapat mengakses dataset dari Kaggle tanpa harus mengunduhnya. Langkah pertama, pastikan kalian sudah memiliki akun [Kaggle](#). Selanjutnya, masuk ke bagian **My Account** dan scroll kebawah hingga menemukan API seperti gambar berikut.



Klik tombol create token dan akan otomatis mengunduh **kaggle.json** yang berisi informasi kredensial untuk mengakses Kaggle API ke local device kalian. Kemudian, pada google colab/jupyter notebook, install library Kaggle :

```
[ ] # Install kaggle
!pip install kaggle
```

Upload file kaggle.json ke colab/jupyter notebook kalian. Setelah itu, pastikan file kaggle.json berada di direktori yang tepat, gunakan command berikut :

```
[ ] # salin kaggle.json ke folder /root/.kaggle/ agar kaggle cli dapat mengaksesnya.
!mkdir /.kaggle
!mv kaggle.json /.kaggle
!mv /.kaggle /root/
!chmod 600 ~/.kaggle/kaggle.json
```

Dan yap, setelah melakukan set-up untuk akun kaggle kalian dan menghubungkan colab ke Kaggle, sekarang kalian bisa mengakses dataset dari Kaggle tanpa perlu ribet lagi mengunduh datasetnya. Untuk mengunduh dataset dari Kaggle, gunakan command berikut :

```
!kaggle datasets download -d <path-to-dataset> -p <download-location>
```

Mari kita ambil contoh, akses dataset berikut di [Kaggle](#). Kalian akan melihat tombol download pada bagian kanan atas. Klik tombol tersebut kemudian download via Kaggle CLI dan copy command yang sudah disediakan seperti kode berikut.

```
[ ] #!/bin/bash
!kaggle datasets download ulrikthgepedersen/fastfood-nutrition
!unzip fastfood-nutrition.zip
```

Dataset yang sudah di download akan masuk ke local colab kalian dalam bentuk **.zip** folder. Kemudian, unzip file tersebut menggunakan command diatas. Dengan demikian, kalian tidak perlu mengunduh dataset secara manual dan tidak perlu menghabiskan storage kalian 🤗🎉

C. Inisialisasi dataset dari google drive

Jika kalian harus mengunduh dataset, kalian bisa menaruh dataset tersebut pada akun google drive. Untuk inisialisasi, gunakan kode berikut:

```
from google.colab import drive
drive.mount("/content/drive")
```

Kemudian, pastikan dataset sudah kalian upload pada google drive. Untuk membaca dataset yang akan digunakan, copy path dataset kemudian gunakan pandas untuk membaca dataset kalian.

```
[ ] df_from_drive = pd.read_csv('/content/drive/MyDrive/Dataset/fastfood.csv') # Ubah sesuai dengan path dataset di drive kalian
```

Dengan cara ini, jika kalian keluar dari colab dan perlu melakukan running ulang, kalian tidak perlu mengupload dataset secara manual berulang kali.

2. Data Cleaning

Data cleaning adalah proses mendeteksi dan mengoreksi data yang tidak akurat atau data yang aneh dari kumpulan data, tabel, atau database untuk mengidentifikasi bagian data yang tidak lengkap, tidak benar, tidak akurat atau tidak relevan, tidak rapi, untuk kemudian dimodifikasi, diganti, atau dihapus sesuai kebutuhan.

Terdapat berbagai macam data yang sering dijumpai, ada data yang telah bersih dan siap untuk diolah, namun ada pula data yang kotor dengan indikasi terdapat data yang hilang, value data yang tidak sesuai dengan tipe data, dan data yang ganda atau duplikasi. Keberadaan data yang masih kotor akan membawa impact yang besar untuk hasil analisis kalian, dan tidak jarang akan menyebabkan kesalahan interpretasi hasil ketika melakukan agregasi.

A. Handling Missing Value

Missing value merupakan kejadian yang umum terjadi ketika mengolah data, dikarenakan data yang tidak tersedia sehingga terdapat beberapa data yang hilang. Missing value dapat dideteksi dengan munculnya nilai NaN (Not Available) atau blank space pada suatu tabel. Mari kita coba.

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol	sodium	total_carb	fiber	sugar	protein	vit_a	vit_c	calcium	salad
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
510	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
511	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False
512	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False
513	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False
514	False	False	False	False	False	False	False	False	False	False	False	False	False	True	True	True	False

515 rows x 17 columns

Dapat kita lihat dari output di atas, apabila value adalah False, maka data tersebut tidak NaN, begitu juga sebaliknya. Apabila value dari suatu data adalah True, maka data tersebut NaN atau missing value. Untuk menampilkan jumlah data NaN, kita akan

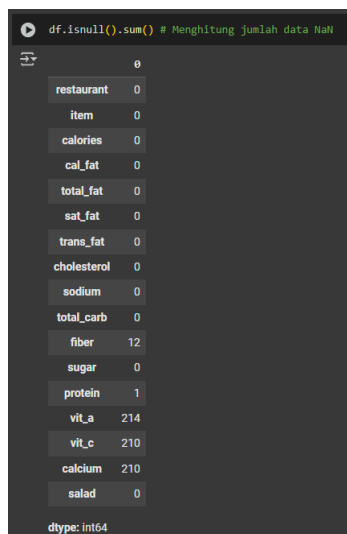
menambahkan syntax **sum()** seperti contoh berikut. Angka pada masing - masing kolom menampilkan jumlah missing value pada kolom dataset.

Dari output di atas, kita mendapat informasi bahwa beberapa kolom memiliki data NaN/missing value. Beberapa cara yang dapat kita lakukan adalah sebagai berikut.

a. Menghilangkan atau menghapus data

```
df.dropna(axis=0 , inplace=True)
```

Fungsi dropna() akan menghapus data yang memiliki nilai NaN. Parameter axis=0 akan menentukan bahwa baris dengan nilai NaN akan dihapus, sedangkan axis=1 akan menghapus kolom yang memiliki nilai NaN. Selanjutnya, parameter inplace=True memastikan bahwa perubahan dilakukan langsung pada DataFrame asli tanpa perlu menyimpan hasilnya ke variabel baru, sementara nilai default inplace=False hanya akan



```
df.isnull().sum() # Menghitung jumlah data NaN
```

	0
restaurant	0
item	0
calories	0
cal_fat	0
total_fat	0
sat_fat	0
trans_fat	0
cholesterol	0
sodium	0
total_carb	0
fiber	12
sugar	0
protein	1
vit_a	214
vit_c	210
calcium	210
salad	0

dtype: int64

membuat salinan DataFrame yang telah dimodifikasi tanpa mengubah DataFrame asli.

```
[ ] # Dari dataset diatas, kita akan mencoba untuk menghapus data yang tidak kita butuhkan untuk analisis
df.dropna(subset=['fiber'], inplace=True) # subset digunakan untuk menentukan kolom yang akan diperiksa
```

Selanjutnya, kita dapat menghapus kolom yang tidak dibutuhkan menggunakan syntax berikut :

```
df.drop(['column_name'], axis=1)
```

Dengan menghapus kolom yang memiliki missing value terbanyak akan memudahkan kita untuk melakukan analisis data. Tetapi, jika kita membutuhkan kolom tersebut untuk analisis, apakah harus tetap dihapus? tentu tidak.

b. Mengisi data NaN menggunakan Data Actual

Langkah selain menghapus data pada suatu kolom, kita dapat melakukan inputting value dengan data yang menyerupai seperti minimum value, maximum value, mean, atau modus. Perhatikan langkah - langkah berikut. Pertama, kita tampilkan deskripsi statistik dari dataset.

```
[ ] df.describe()
```

	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol	sodium	total_carb	fiber	sugar	protein	vit_a	vit_c	calcium
count	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	503.000000	301.000000	305.000000	305.000000
mean	524.453280	234.168986	26.075547	7.983101	0.453280	71.123260	1234.870775	45.447316	4.137177	7.198807	27.709742	18.857143	20.170492	24.852459
std	280.173625	163.384648	18.074278	6.273975	0.827385	61.979954	689.702021	25.082190	3.037460	6.798189	17.714290	31.384330	30.592243	25.522073
min	20.000000	0.000000	0.000000	0.000000	0.000000	0.000000	15.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000
25%	330.000000	120.000000	14.000000	4.000000	0.000000	35.000000	800.000000	28.000000	2.000000	3.000000	15.000000	4.000000	4.000000	8.000000
50%	480.000000	200.000000	23.000000	7.000000	0.000000	60.000000	1100.000000	44.000000	3.000000	6.000000	24.000000	10.000000	10.000000	20.000000
75%	680.000000	310.000000	34.000000	10.000000	1.000000	90.000000	1540.000000	56.000000	5.000000	9.000000	35.000000	20.000000	30.000000	30.000000
max	2430.000000	1270.000000	141.000000	47.000000	8.000000	805.000000	6080.000000	156.000000	17.000000	87.000000	186.000000	180.000000	400.000000	290.000000

Kemudian, gunakan kode berikut. Dengan menggunakan fungsi **fillna()**, kita dapat mengisi kolom pada DataFrame yang memiliki nilai NaN.

```
df['column_name'].fillna(int(df['column_name'].mean()), inplace=True)
```

```
[9] # Dari dataset diatas, kita akan memberikan value pada nilai NaN untuk masing - masing kolom
for col in ['fiber', 'vit_a', 'vit_c', 'calcium']:
    df[col] = df[col].fillna(int(df[col].mean()))

# Menampilkan data setelah memberikan value
df.isnull().sum()
```

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol	sodium	total_carb	fiber	sugar	protein	vit_a	vit_c	calcium	salad
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0

Voilaaa, data yang tadinya memiliki ratusan nilai NaN, kini sudah bersih dari nilai NaN dan dapat kita lanjutkan untuk langkah selanjutnya. For your information, kita juga dapat melakukan *manual inputting value* dengan data yang sebenarnya (actual data) atau *custom value* berdasarkan asumsi bisnis yang terjadi. Tetapi tentu memerlukan waktu dan effort yang banyak untuk mencari data yang hilang dan menginputnya satu persatu.

B. Duplicate Data

Duplikasi data (duplicate) terjadi ketika terdapat nilai atau ID yang berulang pada suatu data. Selain itu, suatu data dikatakan duplikat apabila terdapat informasi yang sama dalam setiap kolom pada data tersebut. Permasalahan duplikasi ini perlu dilakukan data cleaning, karena akan menyebabkan double perhitungan ketika dilakukan suatu analisis lebih mendalam.

Langkah pertama yang dilakukan yaitu mengecek apakah terdapat duplikasi pada data, biasanya dapat diindikasikan dengan munculnya value yang sama pada semua kolom. Jika dalam data Anda terdapat kolom yang dengan informasi yang unique atau tidak boleh double. Cara untuk melakukan pengecekan duplikasi data dapat menggunakan syntax berikut.

- Untuk memeriksa jumlah data yang terduplikasi :

```
df.duplicated.sum()
```

- Untuk memeriksa data duplikat pada semua kolom :

```
df[df.duplicated(keep=False)]
```

- Untuk memeriksa data duplikat pada kolom tertentu :

```
df[df.column_name.duplicated(keep=False)].sort_values('column_name')
```

- Untuk menghapus data duplikat pada satu baris dataframe

```
df.drop_duplicates(keep='first', inplace=True)
```

Parameter keep:

- keep= 'first': Menyimpan baris pertama dari duplikasi dan menghapus sisanya (default).
- keep= 'last': Menyimpan baris terakhir dari duplikasi dan menghapus sisanya.
- keep= False: Menghapus semua baris yang terduplikasi.

- Selanjutnya, untuk menghapus data duplikat pada kolom tertentu

```
df.drop_duplicates(subset=['column_name']).reset_index()
```

3. Data Manipulation

Data manipulation adalah suatu proses memanipulasi atau mengubah data agar mudah diorganisir dan mudah untuk dibaca/dipahami. Data manipulation akan sangat membantu dalam proses analisis data. Ada beberapa contoh penerapan data manipulation yang akan dijelaskan sebagai berikut.

A. Memeriksa dan Mengubah Tipe Data

Fungsi dtypes dan astype() pada Pandas dapat digunakan untuk memeriksa dan mengubah data type. Hal ini dapat memudahkan dalam menganalisa suatu data.

- Gunakan syntax berikut untuk memeriksa data type dari suatu kolom

```
df.<nama_kolom>.dtypes
```

- Untuk mengubah data type pada suatu kolom, kalian bisa menggunakan syntax berikut

```
df['nama_kolom_baru'] df['nama_kolom yang akan diubah'].astype(tipe_data)
```

B. Mengganti Nama Kolom

Untuk mengubah nama kolom seperti pada contoh di bawah ini. Hal ini akan dibutuhkan apabila dari data ternyata nama-nama kolom sulit dipahami konteksnya atau terlalu panjang/pendek.

- Untuk mengubah nama kolom dalam DataFrame, contoh syntax yang bisa kalian gunakan :

```
df.rename(columns={'A': 'X', 'B': 'Y', 'C': 'Z'}, inplace=True)
```

C. Menghapus Data

Tidak semua data digunakan dalam analisa. Semakin banyak data yang digunakan akan semakin mempersulit kita dalam menganalisa, sehingga kita perlu untuk menghapus beberapa data yang tidak kita gunakan.

- Untuk mereduksi data, kalian bisa menggunakan perintah drop seperti berikut:

```
df.drop('Z', axis=1, inplace=True)
```

4. Handle Outliers

Outlier merupakan suatu observasi pada kumpulan data yang memiliki pola yang berbeda dari observasi-observasi lainnya pada kumpulan data tersebut. Artinya, terdapat nilai yang jauh lebih kecil atau jauh lebih besar dibandingkan data lainnya dalam dataset. Penyebab munculnya outlier antara lain:

- Adanya kesalahan prosedur dalam menginputkan data
- Kesalahan dalam pengukuran atau analisis
- Adanya keadaan khusus outlier

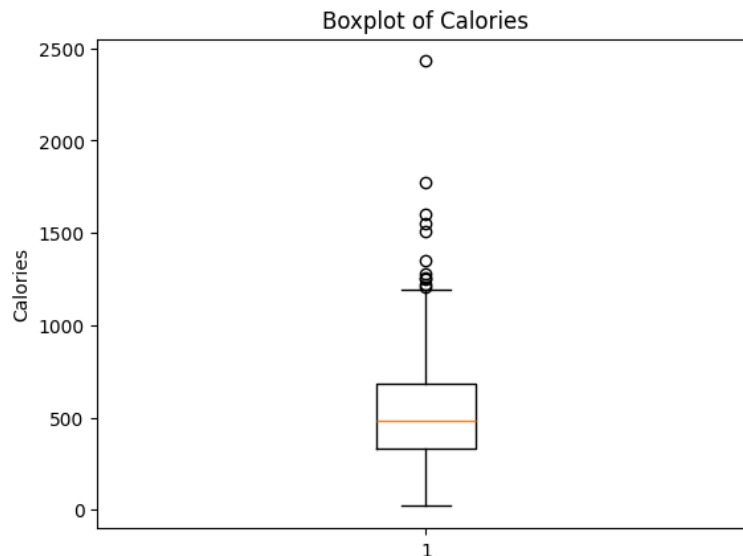
Lalu, mengapa penting untuk mengidentifikasi outliers? Outliers dapat memengaruhi mean pada data dan membuat hasil analisis tidak akurat. Selain itu, kita dapat memeriksa apakah

ada kesalahan pada data yang perlu diperbaiki. Kadang, outliers adalah data penting yang mewakili situasi ekstrem.

Berikut merupakan beberapa cara mendeteksi outlier :

A. Boxplot

Boxplot adalah metode visual yang efektif untuk mengidentifikasi outliers. Outliers seringkali ditampilkan sebagai titik-titik yang berada di luar "whiskers" boxplot. Mari kita coba untuk menampilkan outliers pada salah satu kolom DataFrame.



Hasil visualisasi di atas dapat kita analisa sebagai berikut :

1. Kotak tengah (box) mewakili data antara kuartil pertama (Q1) dan kuartil 3 (Q3). Sedangkan garis di dalam kotak adalah median. Data calories menunjukkan mayoritas nilai kalori berada antara 250 hingga sekitar 750.

2. Whiskers (Garis Horizontal) menunjukkan rentang data "normal" tanpa outliers. Data calories menunjukkan batas atas data normal ada di sekitar 1000.
3. Titik di atas atau bawah whiskers merupakan **outliers**. Titik - titik ini berada di luar batas normal (lebih kecil dari lower bound atau lebih besar dari upper bound). Pada visualisasi di atas, outliers berada diatas 1000. Hal ini berarti bahwa ada makanan dengan kalori lebih dari 1500 hingga 2500, yang jauh lebih besar dari data lainnya.

B. Z-Score

Z-Score mengukur seberapa jauh suatu pengamatan dari mean, dalam satuan standar deviasi. Dengan kata lain, z-score menunjukkan apakah sebuah nilai itu :

1. Di atas rata-rata (z-score positif).
2. Di bawah rata-rata (z-score negatif).
3. Sama dengan rata-rata (z-score = 0)

Dengan menggunakan Z-Score, kita dapat mendeteksi outliers suatu data. Data dengan $z > 3$ atau $z < -3$ sering dianggap sebagai outliers. Z-Score mengubah data menjadi distribusi dengan rata-rata 0 dan standar deviasi 1, sehingga cocok untuk algoritma *machine learning*.

```
[ ] from scipy import stats

z_scores = np.abs(stats.zscore(df['calories']))
outliers = np.where(z_scores > 3)

print("Indeks outliers:", outliers)
```

Indeks outliers: (array([38, 39, 44, 47, 190]),)

C. Interquartile Range (IQR)

Cara lain untuk mendeteksi outliers adalah dengan menggunakan Interquartile Range (IQR). IQR fokus ke kuartil data, yaitu pembagian data menjadi empat bagian yang sama besar

$$\text{IQR} = Q3 - Q1$$

- Q1 (Kuartil Pertama): Nilai di mana 25% data berada di bawahnya.
- Q3 (Kuartil Ketiga): Nilai di mana 75% data berada di bawahnya.
- IQR: Jarak antara Q3 dan Q1, menunjukkan rentang data utama.

Kita dapat menggunakan IQR untuk mendeteksi outliers apabila data memiliki distribusi yang tidak normal. IQR efektif untuk mendeteksi data yang jauh di luar rentang utama.

```
[ ] Q1 = np.percentile(df['calories'], 25)
    Q3 = np.percentile(df['calories'], 75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = np.where((df['calories'] < lower_bound) | (df['calories'] > upper_bound))
    print("Indeks outliers:", outliers)
```

Indeks outliers: (array([37, 38, 39, 44, 47, 96, 99, 113, 190, 206, 258]),)

B. Data Visualization

Ada banyak sekali jenis visualisasi data, dari yang sederhana hingga yang kompleks. Setiap jenis visualisasi biasanya lebih tepat digunakan untuk suatu tujuan tertentu. Secara garis besar, ada empat tujuan visualisasi secara umum.

1. Perbandingan/Komparasi

Jika ingin membandingkan suatu sekumpulan nilai dengan nilai lainnya, ingin melihat mana yang lebih besar, mana yang lebih tinggi, mana yang naik, dsb. Untuk data yang terkait dengan waktu, visualisasi yang digunakan biasanya berupa line-chart. Sedangkan, jika yang dibandingkan adalah kategorikal, maka lebih cocok menggunakan bar-chart.

2. Distribusi

Apabila ingin melihat distribusi/persebaran data di suatu variabel. Visualisasi yang lazim digunakan adalah histogram. Bisa melihat apakah datanya banyak terkumpul di nilai-nilai kecil, berat di kanan, atau simetris, ataupun berdistribusi normal. Untuk melihat distribusi di dua variabel, scatterplot juga bisa digunakan.

3. Komposisi

Digunakan untuk melihat komposisi dari suatu variabel, jika di-breakdown terhadap suatu dimensi data. Visualisasi yang biasa digunakan adalah stacked bar-chart untuk data kategorikal, atau stacked line-chart untuk data terkait waktu.

4. Keterhubungan/Relasi

Ketika ingin melihat keterhubungan antara suatu variabel dengan variabel lain. Misalnya ingin cek, jika variabel A semakin tinggi, apakah variabel B juga semakin tinggi, atau justru lebih rendah, atau tidak ada keterhubungannya. Visualisasi yang biasa digunakan untuk tujuan ini adalah scatter-plot.

Berikut macam-macam contoh visualisasi data dasar yang sering digunakan.

A. Bar Chart

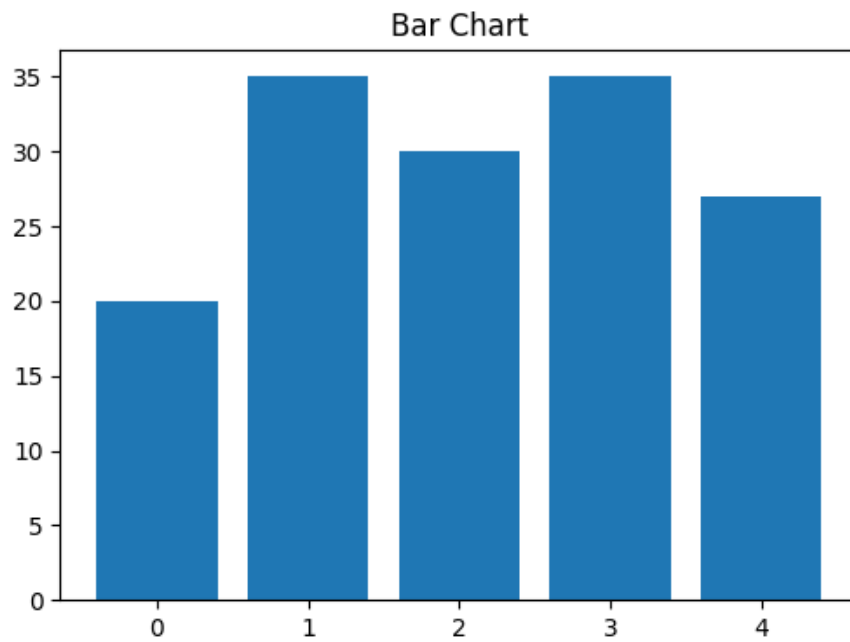
Syntax :

```
import matplotlib.pyplot as plt
import numpy as np

data_bar = [20, 35, 30, 35, 27]

plt.figure(figsize=(13, 9))
plt.subplot(2, 2, 1) # row 2, col 2, position 1
plt.bar(range(len(data_bar)), data_bar)
plt.title('Bar Chart')
```

Output :



B. Multi Bar Chart

Syntax :

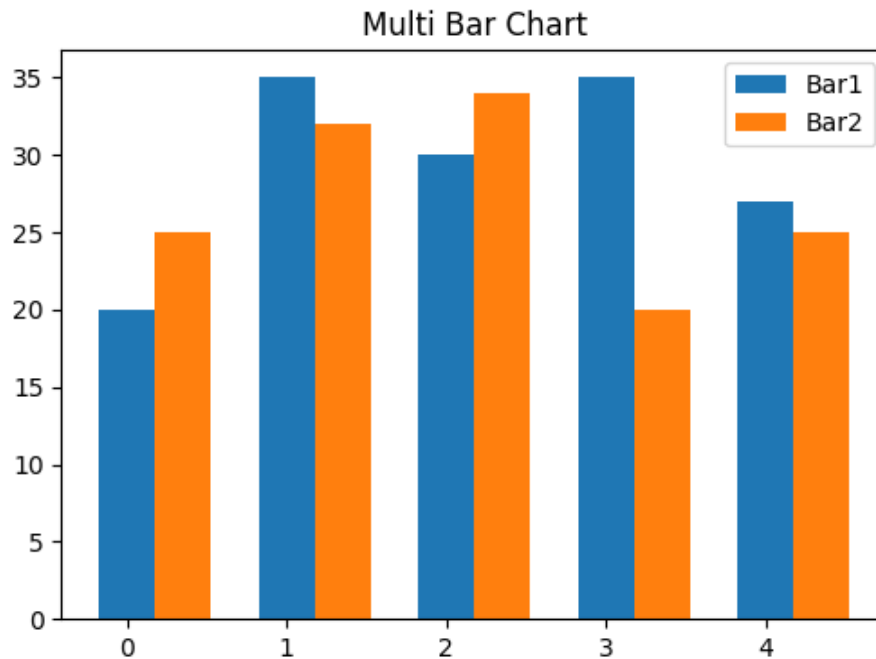
```
[14] import matplotlib.pyplot as plt
import numpy as np

multi_data_bar = {'Bar1': [20, 35, 30, 35, 27],
                  'Bar2': [25, 32, 34, 20, 25]}

plt.figure(figsize=(13, 9)) # Menentukan ukuran visualisasi
plt.subplot(2, 2, 2) # row 2, col 2, posisi 2
bar_width = 0.35
index = np.arange(len(multi_data_bar['Bar1']))

plt.bar(index, multi_data_bar['Bar1'], bar_width, label='Bar1')
plt.bar(index + bar_width, multi_data_bar['Bar2'], bar_width, label='Bar2')
plt.title('Multi Bar Chart')
plt.legend()
plt.show()
```

Output :



C. Line Chart

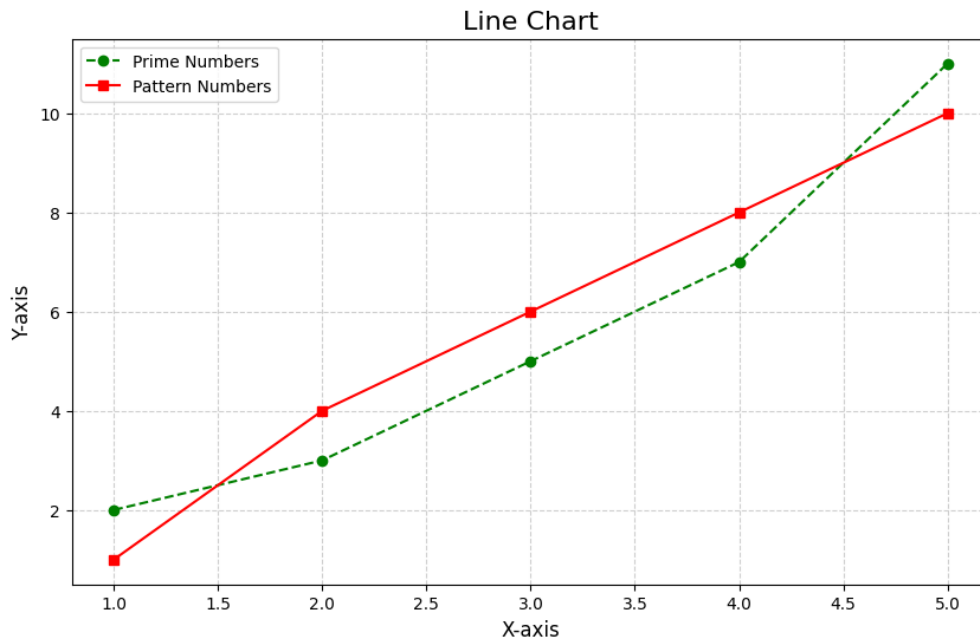
Syntax :

```
# Data
x = [1, 2, 3, 4, 5]
y1 = [2, 3, 5, 7, 11]
y2 = [1, 4, 6, 8, 10]

plt.figure(figsize=(10, 6))
plt.plot(x, y1, label='Prime Numbers', color='green',
         marker='o', linestyle='--')
plt.plot(x, y2, label='Pattern Numbers', color='red',
         marker='s')
plt.title('Line Chart', fontsize=16)
plt.xlabel('X-axis', fontsize=12)
plt.ylabel('Y-axis', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(fontsize=10)

plt.show()
```

Output :



D. Histogram

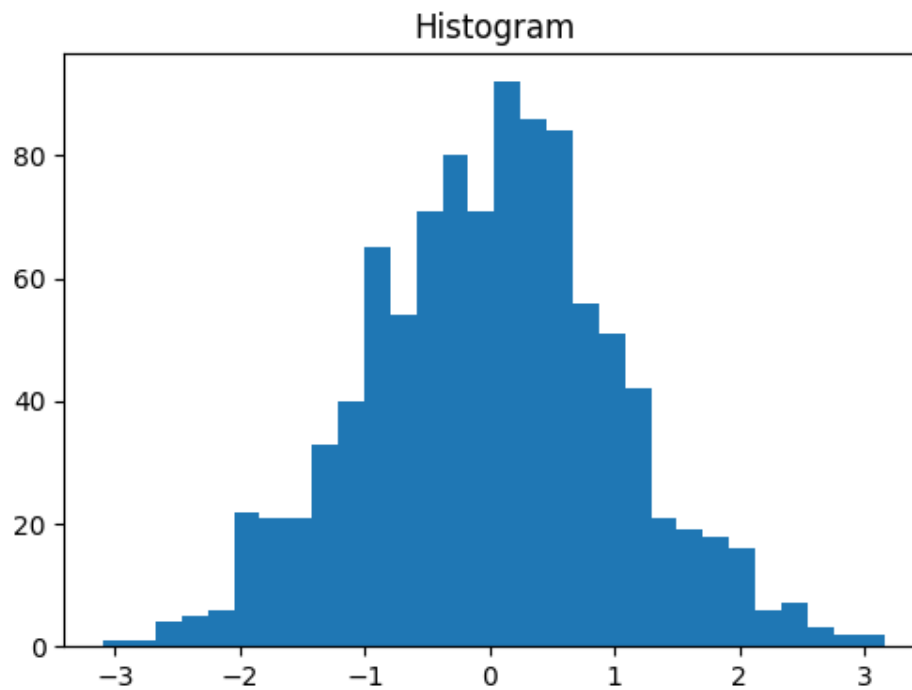
Syntax :

```
[16] import matplotlib.pyplot as plt
import numpy as np

hist_data = np.random.normal(0, 1, 1000) # Menghasilkan angka acak mengikuti distribusi normal

plt.figure(figsize=(13, 9))
plt.subplot(2, 2, 3)
plt.hist(hist_data, bins=30)
plt.title('Histogram')
```

Output :



E. Scatterplot

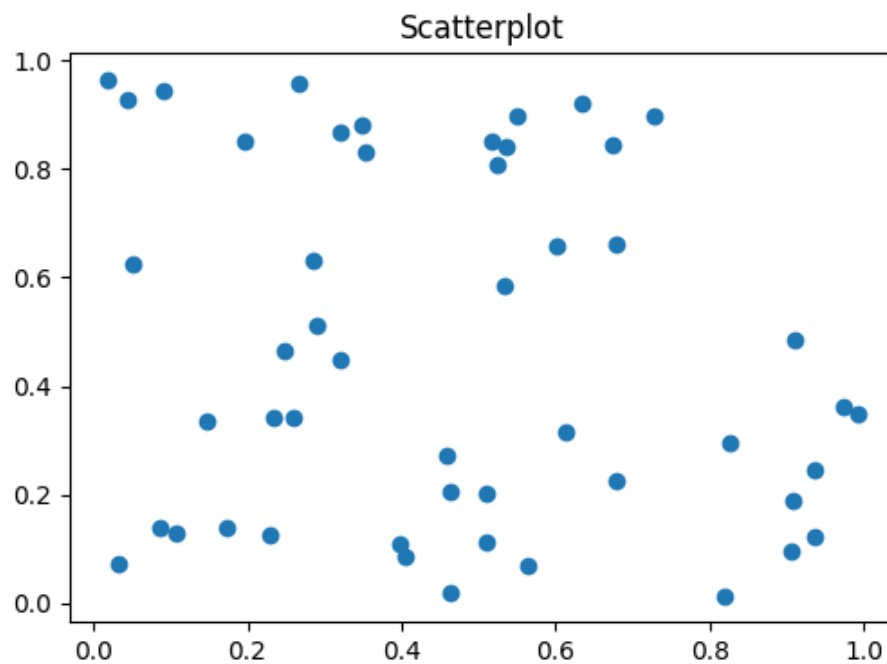
Syntax :

```
[17] import matplotlib.pyplot as plt
import numpy as np

scatter_data_x = np.random.rand(50)
scatter_data_y = np.random.rand(50)

plt.figure(figsize=(13, 9))
plt.subplot(2, 2, 4) # row 2, col 2, position 4
plt.scatter(scatter_data_x, scatter_data_y)
plt.title('Scatterplot')
```

Output :



LATIHAN PRAKTIKUM

Akses dataset [disini](#). Kemudian lakukan insight dasar dari dataset tersebut.

1. Memuat dataset dengan menggunakan library **pandas** menjadi dataframe. **(2 point)**
2. Tampilkan 7 baris data awal dan tampilkan informasi dasar dataset. **(2 point)**
3. Hitung jumlah iuran dan tentukan rata-rata iuran dari semua anggota **(6 point)**

TUGAS PRAKTIKUM

Dataset :

- Akhir NIM Ganjil (1,3,5,7,9) : [Cafe Sales](#)
- Akhir NIM Genap (0,2,4,6,8) : [Music Tours](#)

Setiap data analyst, pasti memiliki cara tersendiri untuk mencapai tujuan dari analisisnya. Dari dataset yang sudah diberikan, cobalah analisis data diatas dan buatlah visualisasi data supaya client dapat memahami point penting dari data yang sudah kalian olah.

Kegiatan 1 - EDA

1. Menampilkan metadata dataset secara lengkap disertai penjelasan ringkas **(6 point)**
2. Data cleaning dan data manipulation **(6 point)**
3. Menganalisis business problem/question, minimal 5 analisis **(6 point)**
4. Menampilkan analisis deskriptif statistik / data outlier **(6 point)**

Kegiatan 2 - Visualisasi

1. Membuat visualisasi data sesuai kategori, bisa comparison/ relationship/ distribution sebanyak 3 grafik. Jelaskan dari hasil visualisasi tersebut kepada asisten. **(6 point)**

Apabila tidak sesuai dengan kriteria tugas dan source code/langkah - langkah analisis persis/mirip dengan praktikan lain, maka akan dilakukan pengurangan nilai bahkan nilai maksimal D.

KRITERIA & DETAIL PENILAIAN

Detail	Bobot Nilai
Pemahaman Materi dan Ketepatan Jawaban	60
Kegiatan 1 dan Kegiatan 2	30
Latihan Praktikum	10